

Getting Started

Install & Run

npm (recommended)

```
$ npm install -g @openai/codex
$ npm install -g @openai/codex@latest # upgrade
```

Homebrew (macOS)

```
$ brew install codex
```

First run

```
$ codex # launch interactive TUI
$ codex "Explain this codebase" # with initial prompt
```

On first run you'll be prompted to sign in via ChatGPT or an API key.

Shell completions

```
# Zsh - add to ~/.zshrc
$ eval "$(codex completion zsh)"

# Bash - add to ~/.bashrc
$ eval "$(codex completion bash)"

# Fish
```

Platform Support

Platform	Status
macOS	Stable
Linux	Stable
Windows (WSL)	Recommended
Windows (native)	Experimental

Authentication

```
$ codex login # ChatGPT OAuth
$ codex login --device-auth # device flow
```

```
# Pipe API key from env
$ printenv OPENAI_API_KEY | codex login --with-api-key
$ codex login status          # check auth state
$ # ...
```

Subcommands

Command Reference			
Command	Alias	Status	Description
<code>codex</code>		Stable	Launch interactive TUI
<code>codex exec</code>	<code>codex e</code>	Stable	Run non-interactively (scripting)
<code>codex review</code>		Stable	Non-interactive code review
<code>codex resume</code>		Stable	Resume a previous session
<code>codex fork</code>		Stable	Fork a session into new thread
<code>codex apply</code>	<code>codex a</code>	Stable	Apply Cloud task diff locally
<code>codex login</code>		Stable	Authenticate Codex
<code>codex logout</code>		Stable	Remove stored credentials
<code>codex completion</code>		Stable	Generate shell completions
<code>codex features</code>		Stable	Manage feature flags
<code>codex app</code>		Stable	Launch Codex desktop app
<code>codex mcp</code>		Experimental	Manage MCP servers
<code>codex mcp-server</code>		Experimental	Run Codex as an MCP server

Command	Alias	Status	Description
<code>codex cloud</code>		Experimental	Browse and exec Cloud tasks
<code>codex sandbox</code>		Experimental	Run cmds inside sandbox
<code>codex debug</code>		Experimental	Debugging tools

Exec (Non-interactive)

```
# Basic non-interactive run
$ codex exec "Fix the race condition"
$ codex e "Add unit tests for utils"

# Prompt from stdin
$ echo "Explain main.go" | codex exec -

# Prompt-plus-stdin (0.118+): pipe data AND pass a prompt
$ cat error.log | codex exec "Diagnose this log" -

# Output options
$ codex exec --json "Audit security"
$ codex exec -o last.txt "Summarize"

# Resume a previous exec session
$ codex exec resume --last "Fix the bugs"
$ codex exec resume <SESSION_ID> "Implement"
```

Global Options

All Flags

Flag	Values	Description
<code>-m, --model</code>	string	Model override (e.g. <code>gpt-5.4</code>)
<code>-s, --sandbox</code>	see below	Sandbox policy
<code>-a, --ask-for-approval</code>	see below	Approval policy
<code>--full-auto</code>	flag	workspace-write + on-request

Flag	Values	Description
<code>--yolo</code>	flag	Bypass all approvals/sandbox
<code>-i, --image</code>	path	Attach image(s) to initial prompt
<code>-c, --config</code>	<code>key=val</code>	Override a config value
<code>-C, --cd</code>	path	Agent working directory
<code>-p, --profile</code>	string	Config profile name
<code>--search</code>	flag	Enable live web search
<code>--add-dir</code>	path	Grant extra directory write access
<code>--oss</code>	flag	Use local OSS provider
<code>--local-provider</code>	lmstudio/ollama	Specify OSS provider
<code>--enable</code>	feature	Enable a feature flag
<code>--disable</code>	feature	Disable a feature flag
<code>--no-alt-screen</code>	flag	Inline TUI mode (no alt screen)
<code>--remote</code>	<code>ws://host:port</code>	Connect to remote app server
<code>--remote-auth-token-env</code>	<code>ENV_VAR</code>	Bearer token env var for remote WS
<code>-V, --version</code>		Print version
<code>-h, --help</code>		Print help

Usage Examples

```
# Specify model
$ codex -m gpt-5.4 "Refactor the auth module"

# Auto mode (low friction)
$ codex --full-auto "Run tests and fix failures"

# Attach a screenshot
```

```
$ codex -i mockup.png "Implement this UI"

# Override config inline
$ codex -c model="gpt-5.4-mini" "Quick scan"

# Live web search
$ codex --search "Find the latest async syntax"

# Set working directory
$ codex -C ~/projects/api "Review this codebase"
```

Sandbox & Approvals

Sandbox Modes

<code>read-only</code>	Inspect only, no edits without approval
<code>workspace-write</code>	Edit workspace + run local commands
<code>danger-full-access</code>	No restrictions (use in hardened envs)

Approval Policies

<code>untrusted</code>	Ask before all non-trusted commands
<code>on-request</code>	Ask when going beyond sandbox boundary
<code>never</code>	Never ask; all commands run

Presets

<code>--full-auto</code>	<code>workspace-write</code> + <code>on-request</code>
<code>--yolo</code>	No sandbox + No approvals

Use `/permissions` in the TUI to change modes mid-session.

Slash Commands

Session Control

<code>/model</code>	Switch model / reasoning effort
<code>/fast</code>	Toggle GPT-5.4 Fast mode

<code>/permissions</code>	Adjust approval mode
<code>/personality</code>	Set response style
<code>/plan</code>	Enter planning mode
<code>/status</code>	Show model, tokens, config
<code>/compact</code>	Summarize to free context
<code>/clear</code>	Clear chat + start fresh
<code>/new</code>	New chat, keep CLI session
<code>/fork</code>	Fork current conversation
<code>/resume</code>	Resume saved session
<code>/exit / /quit</code>	Exit the CLI

Review & Code	
<code>/review</code>	Launch code reviewer
<code>/diff</code>	Show Git diff
<code>/copy</code>	Copy latest Codex output
<code>/mention</code>	Attach file to conversation
<code>/init</code>	Generate AGENTS.md scaffold

Tools & Debug	
<code>/agent</code>	Switch active agent thread
<code>/mcp</code>	List MCP tools
<code>/apps</code>	Browse connector apps
<code>/experimental</code>	Toggle experimental features
<code>/statusline</code>	Configure TUI footer
<code>/ps</code>	Show background terminals
<code>/sandbox-add-read-dir</code>	Add readable dir (Windows)
<code>/feedback</code>	Send logs to maintainers

/plugins	Browse and manage plugins
/title	Set terminal title for session
/debug-config	Print config diagnostics
/logout	Sign out of Codex

Models

Available Models		
Model	Speed	Best for
gpt-5.4	★★★	Default – coding + reasoning + workflows
gpt-5.4-mini	★★★★★	Subagents, light tasks, higher limits
gpt-5.3-codex	★★★★	Complex software engineering
gpt-5.3-codex-spark	★★★★★	Near-instant iteration (Pro only)

Select a Model

```
$ codex -m gpt-5.4 "Your prompt"
$ codex -m gpt-5.4-mini "Quick task"
$ codex --oss      # local Ollama / LM Studio
# Mid-session: type /model in the TUI
```

Configuration

config.toml Keys

```
# ~/.codex/config.toml

# Model settings
model = "gpt-5.4"
model_provider = "openai"
model_reasoning_effort = "medium" # low | medium | high
```

```
# Sandbox & approvals
sandbox_mode = "workspace-write"
approval_policy = "on-request"

# Web search: cached | live | disabled
web_search = "cached"

# Review model override
review_model = "gpt-5.4"

# TUI appearance
[tui]
theme = "dark"
alternate_screen = true

# Profile example
[profile.ci]
model = "gpt-5.4-mini"
approval_policy = "never"
sandbox_mode = "workspace-write"
```

Use `-p ci` to activate a profile.

Feature Flags

```
$ codex features list
$ codex features enable unified_exec
$ codex features disable shell_snapshot

# Per-session (not persisted)
$ codex --enable unified_exec
$ codex --disable shell_snapshot
```

Config Locations

<code>~/.codex/config.toml</code>	Global defaults
<code>~/.codex/AGENTS.md</code>	Personal instructions
<code>~/.agents/skills/</code>	Personal skills
<code>.agents/skills/</code>	Repo-level skills
<code>AGENTS.md</code>	Repo instructions

Customization

AGENTS.md

Persistent repo instructions, checked in alongside code:

Build & test commands

- Install: `npm install`
- Test: `npm test`
- Lint: `npm run lint`

Conventions

- TypeScript strict mode
- No PII in logs
- Auth middleware on all routes

Review guidelines

- Flag unhandled promise rejections

When to add rules:

Repeated agent mistakes

Recurring PR review feedback

Tag [@codex](#) add to [AGENTS.md](#) in GitHub

Skills

Reusable workflows packaged as [SKILL.md](#):

```
my-skill/
  SKILL.md      ← metadata + instructions
  scripts/     ← optional helper scripts
  references/   ← optional documentation
  assets/       ← optional templates
```

Example [SKILL.md](#):

```
---
name: commit
description: Stage and commit changes
  in semantic groups.
```

1. Stage files in logical groups.
2. Group: feat → test → docs → chore

MCP Servers

```
# Add a stdio server
$ codex mcp add myserver -- npx my-mcp-server

# Add an HTTP server
$ codex mcp add myserver \
  --url https://api.example.com/mcp

# With auth token
$ codex mcp add myserver \
  --url https://api.example.com/mcp \
  --bearer-token-env-var MY_TOKEN

# List / remove
$ codex mcp list
$ codex mcp remove myserver

# Run Codex itself as MCP server
$ codex mcp-server
```

Session Management

Resume Sessions

```
$ codex resume          # session picker
$ codex resume --last   # most recent session
$ codex resume --all    # across all directories
$ codex resume <ID> "Continue fixing tests"
```

Sessions stored at [~/.codex/sessions/](#)

Fork a Session

```
$ codex fork            # fork from picker
$ codex fork --last     # fork most recent
$ codex fork <SESSION_ID> "New direction"
```

Forking preserves the original transcript while starting a new branch.

Image Inputs

```
# Single image
$ codex -i screenshot.png "Fix this error"

# Multiple images
$ codex --image img1.png,img2.jpg "Compare"

# Or drag images into the TUI composer
```

Supported: PNG, JPEG and other common formats.

Cloud & GitHub

Cloud Tasks

```
# Browse cloud environments
$ codex cloud

# Submit a cloud task
$ codex cloud exec --env <ENV_ID> "Fix bug"

# With multiple attempts (best-of-N)
$ codex cloud exec --env <ENV_ID> \
  --attempts 3 "Write failing tests"

# List recent tasks
$ codex cloud list

# Check task status
$ codex cloud status <TASK_ID>

# Show diff for a task
$ codex cloud diff <TASK_ID>

# Apply diff to local working tree
$ codex apply <TASK_ID>
```

GitHub Integration

```
# In a PR comment - trigger code review:
@codex review

# Custom focus:
@codex review for security regressions

# Delegate any task using PR as context:
```

@codex fix the CI failures

Enable in Codex Settings → Code review → your repo. Turn on Automatic reviews to review every new PR.

Subagent Workflows

Offload noisy work to parallel agents to avoid **context pollution** and **context rot**:

```
codex "Review this branch with parallel agents.
Spawn one for security risks, one for test gaps,
and one for maintainability. Wait for all three,
then summarize findings with file references."
```

Use for: exploration, tests, triage, log analysis

Avoid parallel write-heavy work (conflicts)

Each subagent consumes extra tokens

Subagent model guide:

Role	Model	Reasoning
Main coordinator	<code>gpt-5.4</code>	medium
Fast exploration	<code>gpt-5.4-mini</code>	low
Security / logic review	<code>gpt-5.4</code>	high

<code>high</code>	Complex logic, security audits
<code>medium</code>	General tasks (default)
<code>low</code>	Simple or speed-critical tasks

Set in `config.toml`:

```
model_reasoning_effort = "medium"
```

Or inline: `-c model_reasoning_effort="high"`